

Instructor: Date:

Learner: Regional Branch:

Program: Center:

CONTENT

- Let's Start! • Let's Explore! • Let's Practice! • Extension Activity • Target Words • Learning Points



Let's Start!

Let's talk about the C# program!



Learning Outcome:

Understand important information about client software for the identity metasystem in spoken or written documents.

Ask your partner:

- How good are you at using the command-line in a computer?
- Name some characteristics of C# when writing a statement

Can Do Statements:

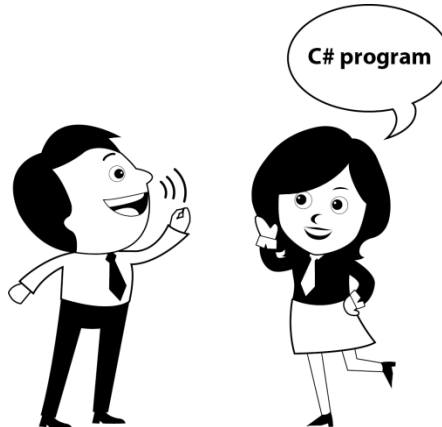
- I can list the parts that compose a C# program.
- I can talk about the different parts of a given program in C#.
- I can say how to compile a C# program by using the command-line.





Let's Explore!

Mr. Anderson is starting a new course on programming.



Instructions: Read the text below.

Mr. Anderson: Welcome to this new lesson on programming. Today we will see the basics of C# program structure. Let's introduce our topic by remembering something Steve Jobs said. He said once that everyone, at least in this country, ought to learn how to program a computer; he said it teaches people how to think. What do you think about programming, Sophie?

Sophie: I believe that programming is greatly important because you can make something completely new from scratch.

Mr. Anderson: Very good answer; you might want to keep it in mind all the time. Well, let's begin with our matter. The parts that compose a C# program are namespace declaration, class, class methods, class attributes, main method, statements and expressions, and comments. These parts can be more or less extensive, depending on the complexity of the application or program we are developing. The namespace encloses one or several classes; it defines the scope of the program, and can also contain more namespaces. You had better name each namespace with a word or phrase that defines the process accurately. The classes contain the definitions of data and methods and their relationships among them. In the comment section, you ought to add some information that tells you some specific information you need when modifying the code; this line doesn't affect the program structure or functioning. You can compile and execute the program by using the command-line or a program. Let's imagine we use Visual Studio for this matter. When using this, you ought to go to the "file" menu, then "new," and then "project." Then, you choose C# and, subsequently, you choose "Windows." Next, you choose console application and set a name for your project. Finally, you write code, you run it and check its correctness and functioning.



Let's Practice!

Let's talk about parts in a C# program.

Instructions: Write the words to correctly complete the sentences. Use words from the box below.

Main method / collection of classes / comments / fields and methods / System namespace declaration

1. _____ is the first part to compose a C# program.
2. There is the _____, which has a print statement.
3. A namespace consists of a _____.
4. Some classes contain _____.
5. Using the correct declaration, I add _____ that don't interfere with the correct functioning of the program.



Let's Practice!

Let's check some useful vocabulary!

Instructions: Match column A with column B.

COLUMN A

1. Operations among classes
2. Parts of the code that don't affect the code functioning
3. In C#, the program file can be different from the...
4. It contains classes
5. It has a print statement which is executed when the program runs
6. You use this to finish a statement in C#

COLUMN B

- a. Namespace
- b. Semicolon
- c. Main method
- d. Class method
- e. Class name
- f. Comments



Extension Activity

I can explain some steps.

Instructions: Complete the following steps. Fill in the Self-Assessment.

- Search for information about C# programs.
- Identify key information related to its structure and steps you might follow to develop a C# program.
- Make a poster showing a list of steps you need to follow and explaining them.
- Share your poster with the class and discuss about the topic.

Self-Assessment

Yes No

- I interpret information about C# programs.
- I list the steps you might follow to develop a C# program
- I produce a poster showing a list of steps one needs to follow.

☐	☐
☐	☐
☐	☐



Target Words

- **subsequently** (adverb): next in a sequence of steps
e.g. Then, you choose C# and, subsequently, you choose "Windows".
- **namespace declaration** (noun): a section of the code in which programmers set a name to determine the scope of determined functions
e.g. The parts that compose a C# program are namespace declaration, class, class methods, class attributes, main method, statements and expressions, and comments.
- **enclose** (verb): to contain or to determine boundaries between some things are
e.g. The namespace encloses one or several classes; it defines the scope of the program, and can also contain more namespaces.
- **accurately** (adverb): in a free-from-error way
e.g. You had better name each namespace with a word or phrase that defines the process accurately.
- **extensive** (adjective): wide, big, or broad
e.g. These parts can be more or less extensive, depending on the complexity of the application or program we are developing.
- **class method** (noun): relationships and operations between classes in a code
e.g. The parts that compose a C# program are namespace declaration, class, class methods, class attributes, main method, statements and expressions, and comments.



Learning Points

Had Better, Ought to, and Might Want to for What Is Advisable

Had better is a modal verb used to say that if you don't do something, you will have a negative result. **Ought to** is used to give advice. **Might want to** is used to recommend something.

Had better, **ought to**, and **might** are modal verbs.

Had better is used to say that if the person doesn't do a certain thing, they will experience a negative consequence.

The structure you use with **had better** is the following:

Subject + **had better** + verb in infinitive + complement.

It is more natural to use **had better** with contraction.

Example: *She'd **better** use lowercase in C#.*

Ought to is similar to should. The difference between them is that **ought to** can't be used in questions.

The structure you use with **ought to** is the following:

Subject + **ought to** + verb in infinitive + complement.

Example: *You **ought to** use a short but accurate name for this namespace.*

Might want to is a polite way of saying that something is a good option. It is also followed by the base form of the verb.

The structure you use with **might want to** is the following:

Subject + **might want to** + verb in infinitive + complement.

Example: *You **might want to** save each project in a different folder.*

Examples

- *Very good answer; you **might want to** keep it in mind all the time.*
- *You **had better** name each namespace with a word or phrase that defines the process accurately.*
- *In the comment section, you **ought to** add some information that tells you some specific information you need when modifying the code.*





Learning Points

Things Not to Forget When Using C#

There are some essential characteristics that will help you write better in C#

These are four important things to remember when writing C# code:

- C# is case sensitive. Remember that you must use upper or lower case depending on the objects and variables; if you don't do it right, the program won't run.
- Finish any statements and expression with a semicolon (;), otherwise, you will have a syntactic error.
- Main method determines when the program execution starts.
- Different from Java, in C#, the program file name can be different from the class name